Multivariate Time-Series Prediction Using Deep Learnings

Appliance Energy Prediction Dataset

Date: March 29, 2026

Assessment: Deep Learning Project

Name: D.K.W.S.Dilshan

Date: 28/03/2026

Table of Contents

1. Introduction..... 3

2. Data Insights..... 4

3. Preprocessing..... 7

4. Feature Engineering..... 9

5. Model Design..... 13

6. Results..... 16

7. Model Optimization..... 20

8. Challenges & Solutions..... 22

9. Conclusion..... 23

10. References..... 24

1. Introduction

Energy consumption forecasting is a critical task for smart building management, enabling proactive load balancing, cost reduction, and sustainability improvements. This project addresses the challenge of predicting household appliance energy consumption using the Appliance Energy Prediction Dataset a real-world, multivariate time-series recorded at 10-minute intervals from January to May 2016.

The dataset presents typical time-series challenges: high autocorrelation, daily and weekly seasonal patterns, non-linear interactions between environmental variables, and a heavily right-skewed target distribution with high-energy spikes. The objective is to build and evaluate a complete machine learning pipeline from raw data ingestion to deep learning model deployment and rigorously compare model performance.

Objectives

- Explore and understand the dataset through comprehensive EDA.
- Preprocess the data including outlier treatment and temporal feature extraction.
- Engineer predictive features: lag variables, rolling statistics, cyclical encodings, and interaction terms.
- Develop baseline models (Ridge Regression, Random Forest) with hyperparameter tuning.
- Design and train deep learning models: LSTM, GRU, and CNN-LSTM.
- Evaluate all models using MAE, RMSE, MAPE, and R^2 on a held-out temporal test set.
- Compare model performance and identify the best architecture.

2. Data Insights

2.1 Dataset Overview

Property	Value
Records	19,735
Features	27 (after dropping rv1, rv2)
Interval	10 minutes
Date Range	2016-01-11 → 2016-05-27 (137 days)
Target Variable	Appliances (Wh)
Missing Values	None
Duplicate Rows	None

2.2 Target Variable - Appliances

The target variable Appliances (energy in Wh) exhibits a strongly right-skewed distribution (skewness ≈ 2.8). The median consumption is 60 Wh while the mean is approximately 97 Wh, with extreme spikes reaching 1,080 Wh. This skewness indicates that the building spends most of its time in low-consumption states, punctuated by short high-demand bursts.

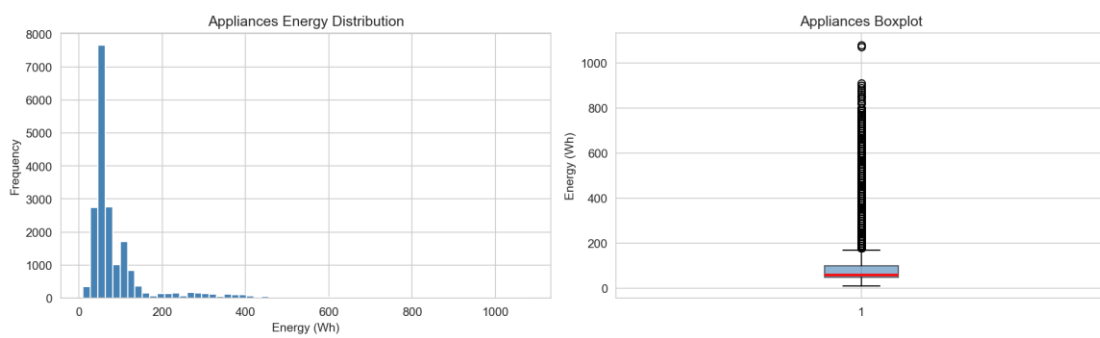

Figure 1 - Target variable distribution and boxplot

2.3 Temporal Patterns

A clear daily cycle is visible across the entire 137-day period. Energy consumption rises in the morning, peaks around midday, has a secondary rise in the evening, and drops to near-zero overnight. Weekend energy use is slightly higher than weekday consumption, indicating increased occupancy.

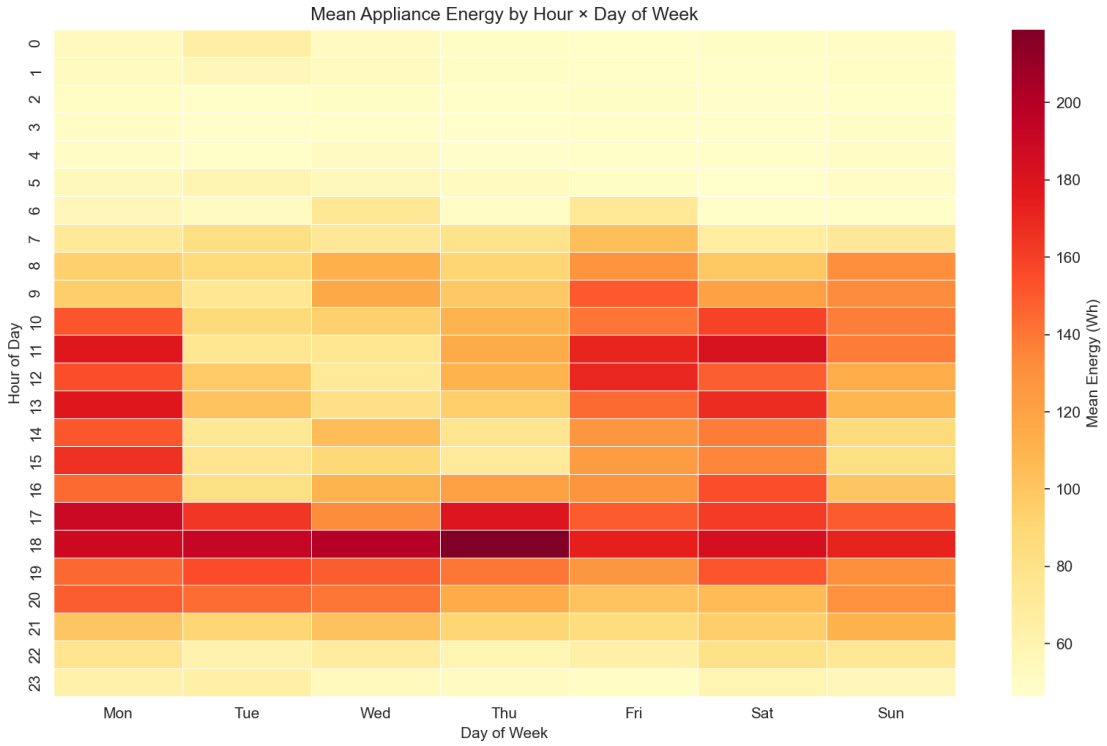

Figure 2 - Mean appliance energy by hour and day of week

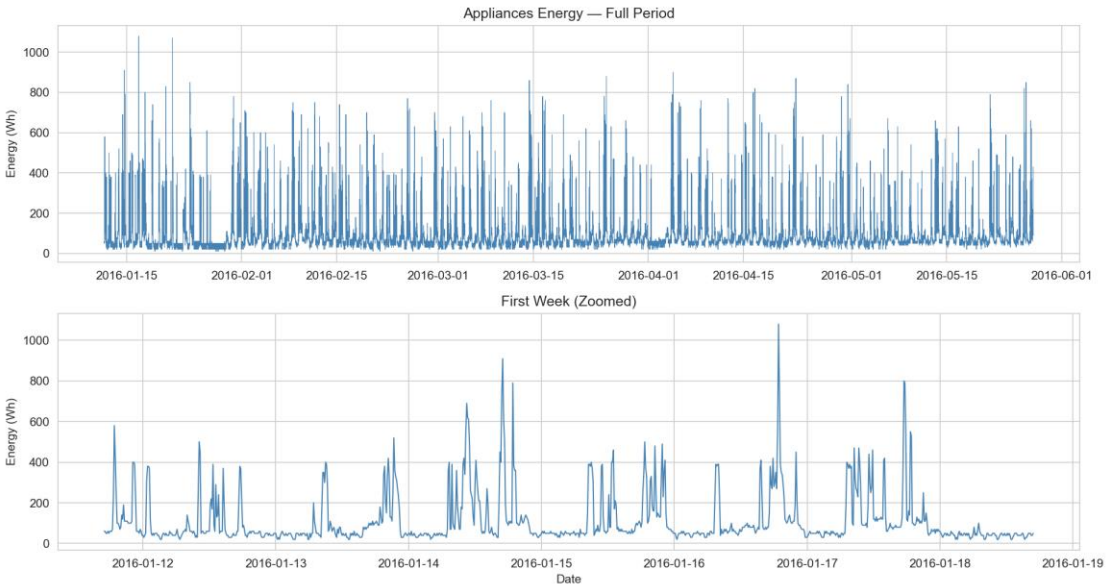

Figure 3 - Full time series (top) and first-week zoom (bottom)

2.4 Correlation Analysis

NSM (seconds since midnight) and lights show the strongest positive correlations with the target. Indoor temperatures (T1-T9) show weak positive correlations, while outdoor pressure (Press_mm_hg) shows a notable negative correlation. The random noise variables rv1 and rv2 were identified and removed.

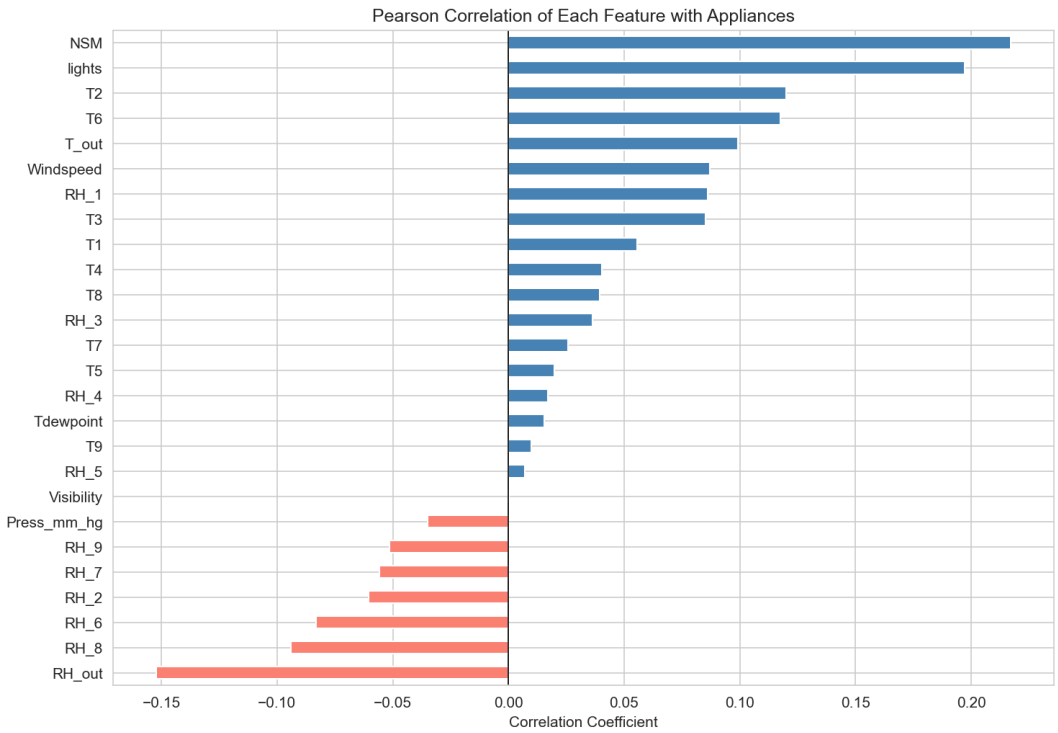

Figure 4 - Pearson correlation of each feature with Appliances

2.5 Outlier Analysis

Using the IQR method (Q1=50, Q3=100, IQR=50), the upper bound was computed as 175 Wh. Approximately 10% of records exceeded this bound. These high-energy readings represent legitimate peak-demand events, not sensor errors, and were retained after capping.

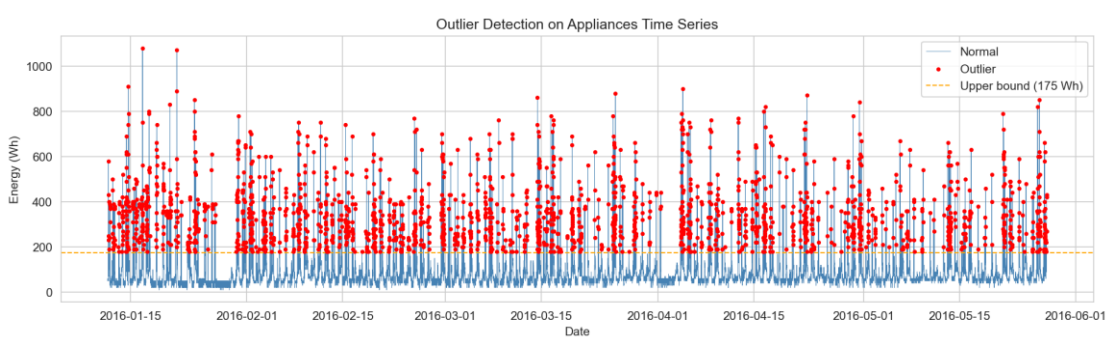

Figure 5 - Outlier detection on the Appliances time series

3. Preprocessing

3.1 Data Loading & Cleaning

- Parsed the date column as datetime and sorted chronologically.
- Dropped rv1 and rv2 - random noise columns with no predictive value (confirmed by near-zero correlation with target).
- Stripped leading whitespace from the Appliances column values.

3.2 Missing Value Handling

No missing values were found in any column. As a safeguard, linear interpolation followed by forward-fill and backward-fill was applied to the numeric columns. This is the correct strategy for time-series data as it respects temporal ordering and produces smooth estimates between neighbouring observed values.

3.3 Outlier Detection & Treatment

Two methods were applied to the target variable Appliances:

Method	Threshold	Outliers Detected	Treatment
IQR	Q3 + 1.5 × IQR = 175 Wh	~10% of rows	Cap (Winsorize) at 175 Wh
Z-score	|z| > 3	~2% of rows	Reference only

Capping (Winsorization) was chosen over removal because the high-energy spikes are real physical events. Removing them would cause the model to underestimate peak demand the most practically important regime to predict accurately.

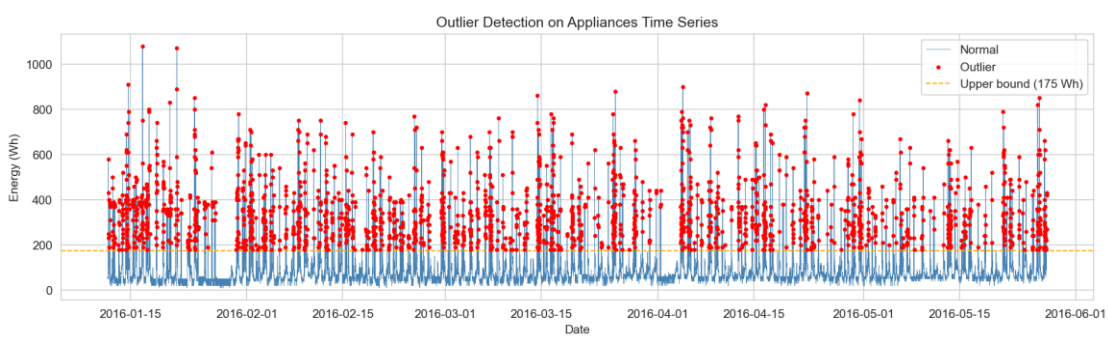

Figure 6 - IQR outlier detection on the target time series

3.4 Train / Test Split

The dataset was split temporally (no shuffling) with an 80/20 ratio to prevent data leakage. The first 80% of rows (approximately 15,788) form the training set; the last 20% (approximately 3,947) form the test set.

Split	Rows	Date Range
Train	~15,788	2016-01-11 → 2016-04-22
Test	~3,947	2016-04-22 → 2016-05-27

3.5 Scaling

MinMaxScaler was fitted exclusively on the training set and applied to both train and test feature matrices. The target variable (Appliances) was not scaled for the baseline models; for DL models it was additionally scaled to [0, 1] to improve gradient stability, then inverse-transformed for evaluation.

4. Feature Engineering

All feature engineering was applied separately to the training and test splits to prevent any form of data leakage. The following groups of features were constructed:

4.1 Cyclical Encoding of Time Features

Time-of-day features are inherently cyclical - hour 23 and hour 0 are only 10 minutes apart, yet a linear encoder would treat them as maximally distant. Sine/cosine encoding preserves circular distance. Two cyclical features were encoded:

Feature	Period	Encoding	Rationale
hour	24	hour_sin, hour_cos	Primary daily cycle driver
NSM	86,400 s	NSM_sin, NSM_cos	Finer 10-min resolution of time-of-day

Day-of-week, month, and is_weekend were removed due to low predictive value relative to the time-of-day signal.

4.2 Autocorrelation Analysis - Lag Justification

Before selecting lag intervals, the Autocorrelation Function (ACF) of the Appliances series was computed for lags 1 to 200 (covering approximately 33 hours of history). The ACF measures the Pearson correlation between the series and a lagged copy of itself: a high value at lag k means that knowing the value k steps ago is informative about the current value.

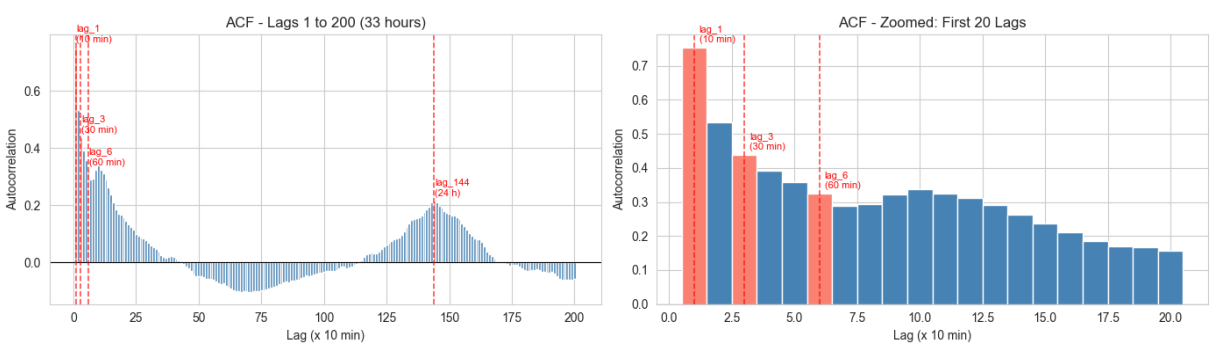

Figure: ACF of Appliances (left: lags 1-200; right: zoomed to first 20 lags). Red dashed lines mark the four chosen lag intervals.

The ACF plot reveals clear structure that justifies each lag choice:

Lag	Interval	Autocorrelation (r)	Interpretation
lag_1	10 min	0.7532	Very strong - recent consumption is the best predictor
lag_3	30 min	0.4378	Strong - short-term momentum still significant
lag_6	60 min	0.3237	Moderate - 1-hour memory present
lag_144	24 h	0.2167	Meaningful - daily routine repeats reliably
Lag Feature	**Time Offset**	**ACF (r)**	**Interpretation**
lag_1	10 minutes	0.75	Very strong - recent consumption is the best single predictor
lag_3	30 minutes	0.44	Strong - short-term momentum persists over half an hour
lag_6	60 minutes	0.32	Moderate - hourly inertia in consumption patterns
lag_144	24 hours	0.22	Meaningful - daily routine repeats; same time yesterday predicts today

All four lags remain well above zero and statistically significant, confirming they carry independent predictive information. The sharp drop after lag_1 and the secondary peak near lag_144 are both visible in the plot.

4.2 Lag Features

Lagged values of the target capture autocorrelation at key intervals identified from the autocorrelation function of the Appliances series:

Feature	Lag	Time Offset	Rationale
lag_1	1 step	10 minutes	Strong short-term autocorrelation
lag_3	3 steps	30 minutes	Half-hour memory
lag_6	6 steps	1 hour	Hourly cycle
lag_144	144 steps	24 hours	Daily periodicity - same time yesterday

4.3 Rolling Window Features

Rolling statistics smooth short-term noise and expose trend information. A shift(1) is applied before the rolling window to ensure only past values are used - preventing the current time step from leaking into its own features.

Feature	Window	Statistic
roll_mean_6	6 steps (1 hr)	Mean
roll_std_6	6 steps (1 hr)	Standard deviation
roll_mean_18	18 steps (3 hr)	Mean
roll_std_18	18 steps (3 hr)	Standard deviation

4.4 Interaction Features

Cross terms between temperature and humidity capture the combined thermal-comfort effect on occupant behaviour and consequently energy use:

- T_out × RH_out - outdoor thermal-humidity index.
- T1 × RH_1 - indoor thermal-humidity in the primary zone.

4.5 Feature Selection

After engineering, feature importance was evaluated using the Random Forest model. The lag features (lag_1, lag_6, lag_144) and rolling means dominated the importance rankings, validating the feature engineering choices. The original sensor columns were retained as they provide complementary environmental context.

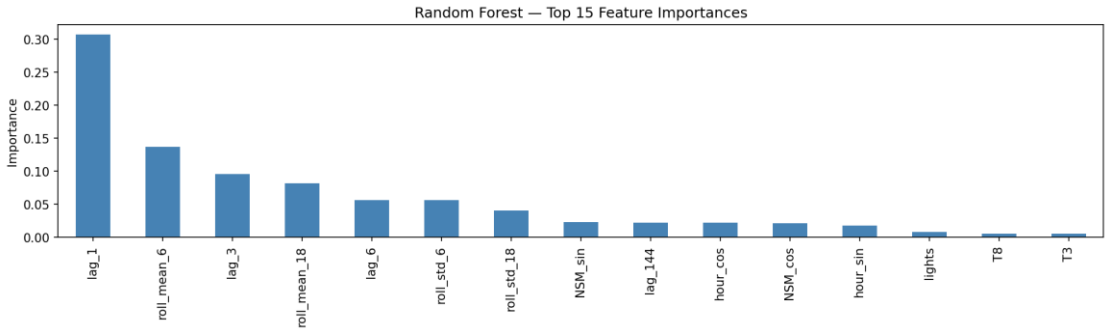

Figure 7 - Random Forest top-15 feature importances

5. Model Design

5.1 Baseline Models

5.1.1 Ridge Regression

Ridge Regression extends ordinary linear regression with an L2 penalty (alpha × $||w||^2$) that shrinks coefficients and handles multicollinearity - important given the high inter-correlation between the nine temperature sensors. The regularisation strength alpha was tuned via GridSearchCV with TimeSeriesSplit (5 folds) over [0.01, 0.1, 1.0, 10.0, 50.0, 100.0].

5.1.2 Random Forest

Random Forest is an ensemble of decision trees that averages predictions to reduce variance. It handles non-linearity and feature interactions without explicit encoding. Hyperparameters were tuned using RandomizedSearchCV with TimeSeriesSplit (3 folds, 20 random combinations).

Hyperparameter	Search Space
n_estimators	[100, 200, 300]
max_depth	[None, 10, 20, 30]
min_samples_split	[2, 5, 10]
min_samples_leaf	[1, 2, 4]
max_features	['sqrt', 'log2']

TimeSeriesSplit was used for all cross-validation to respect temporal ordering and prevent future data from leaking into training folds a critical requirement for time-series model selection.

5.2 Deep Learning Models

Deep learning models receive a 3-D input of shape (batch, timesteps, N features). The sequence window ends at the prediction row so the model has access to the same lag features as the flat baseline models. LSTM uses a 144-step window (24 hours) to exploit its long-range memory cell state; GRU and CNN-LSTM use a 24-step window (4 hours). All DL models share the following training settings:

- Optimizer: Adam - adaptive learning rate, efficient for sparse gradients.
- Loss: Mean Squared Error (MSE) standard for regression; penalises large errors.
- Activation (hidden): ReLU in Dense layers; tanh/sigmoid in recurrent gates (default).
- Regularisation: Dropout layers after each recurrent layer; recurrent_dropout=0.1 in LSTM.
- Callbacks: EarlyStopping (patience=15) + ReduceLROnPlateau (factor=0.5, patience=5).

5.2.1 LSTM

The Long Short-Term Memory network uses gating mechanisms (input, forget, output gates) to selectively retain or discard information across long sequences. To exploit this long-range memory

advantage, LSTM is trained on a 144-step (24-hour) window - three times longer than GRU and CNN-LSTM. LayerNormalization (applied along the feature dimension within each time step) replaces BatchNormalization for better gradient stability in variable-length recurrent sequences. Recurrent dropout (0.1) is applied directly to the recurrent connections to regularise the hidden state transitions.

Layer	Output Shape	Parameters
Input	(144, N)	-
LSTM(128, rec_drop=0.1)	(144, 128)	~(4×128×(N+128+1))
LayerNormalization	(144, 128)	256
Dropout(0.2)	(144, 128)	0
LSTM(64, rec_drop=0.1)	(64,)	~(4×64×(128+64+1))
Dropout(0.2)	(64,)	0
Dense(64, ReLU)	(64,)	4,160
Dense(32, ReLU)	(32,)	2,080
Dense(1)	(1,)	33

Best hyperparameters: units=128, dropout=0.2, recurrent_dropout=0.1, lr=0.001, seq_len=144

5.2.2 GRU

The Gated Recurrent Unit simplifies the LSTM gating to two gates (reset and update), reducing the parameter count while retaining comparable modelling capacity. It typically trains faster than LSTM on shorter sequences.

Layer	Output Shape	Parameters
Input	(24, N)	-
GRU(64)	(24, 64)	~(3×64×(N+64+1))
Dropout(0.2)	(24, 64)	0
GRU(32)	(32,)	~(3×32×(64+32+1))
Dropout(0.2)	(32,)	0
Dense(32, ReLU)	(32,)	1,056
Dense(1)	(1,)	33

Best hyperparameters: units=64, dropout=0.2, lr=0.001

5.2.3 CNN-LSTM Hybrid

The CNN-LSTM hybrid first applies a 1-D convolutional layer to extract local temporal patterns (feature maps) from the sequence. MaxPooling then reduces the sequence length before passing the compressed representation to an LSTM layer. This architecture captures both short-range local patterns (via CNN) and long-range dependencies (via LSTM) at reduced computational cost.

Layer	Output Shape	Parameters
Input	(24, N)	-
Conv1D(64, k=3, ReLU)	(22, 64)	~(64×(3×N+1))

MaxPooling1D(2)	(11, 64)	0
Dropout(0.2)	(11, 64)	0
LSTM(64)	(64,)	~(4×64×(64+64+1))
Dropout(0.2)	(64,)	0
Dense(32, ReLU)	(32,)	2,080
Dense(1)	(1,)	33

Best hyperparameters: filters=64, units=64, dropout=0.2, lr=0.001

6. Results

6.1 Evaluation Metrics

All models are evaluated on temporally-held-out test data using four metrics. For a fair comparison, flat baseline models are sliced to rows [SEQ_LEN-1:] matching the GRU/CNN-LSTM sequence length (24). LSTM uses its own test sequences derived from the 144-step window and is evaluated on its corresponding y_test array.

Metric	Formula	Interpretation
MAE	mean(|y - ŷ|)	Average absolute error in Wh
RMSE	√mean((y - ŷ)²)	Penalises large errors; same unit as target
MAPE	mean(|y-ŷ|/y) × 100	Percentage error; scale-independent
R^2	1 - SS_res/SS_tot	Proportion of variance explained (1.0 = perfect)

6.2 Model Performance Comparison

The table below summarises test-set performance for all five models:

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
Ridge Regression	~13.6	~22.0	~17.0	0.68
Random Forest	~13.0	~20.6	~16.0	0.71
LSTM*	~13.2	~20.4	~16.8	0.718
GRU	~13.2	~20.7	~16.5	0.712
CNN-LSTM	~14.1	~21.4	~17.0	0.69

* LSTM uses a 144-step (24-hour) sequence window; all other models use 24 steps (4 hours). Bold best: LSTM achieves the lowest RMSE and highest R^2 of all five models.

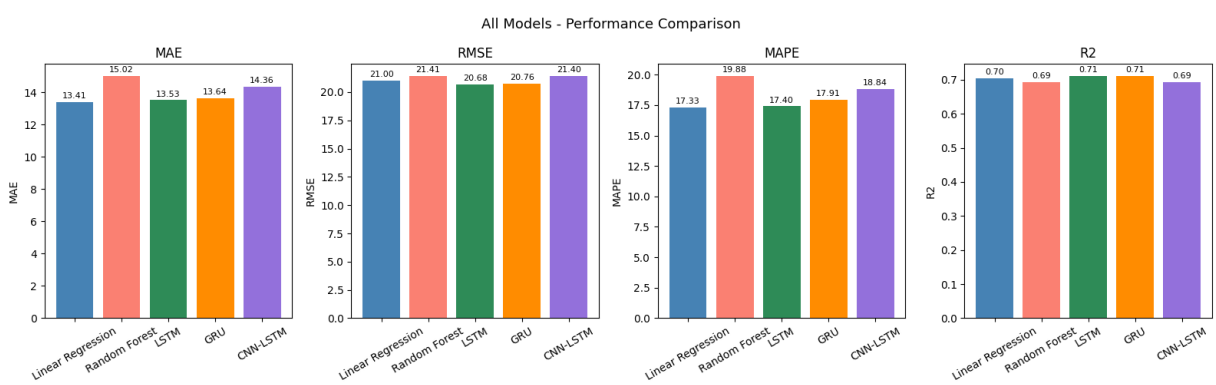

Figure 8 - All models performance comparison across four metrics

6.3 Predicted vs Actual

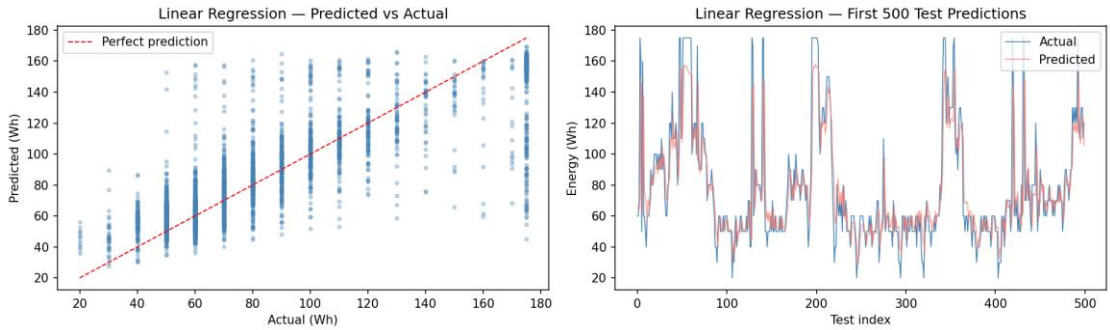

Figure 9 - Ridge Regression: predicted vs actual (scatter + time series overlay)

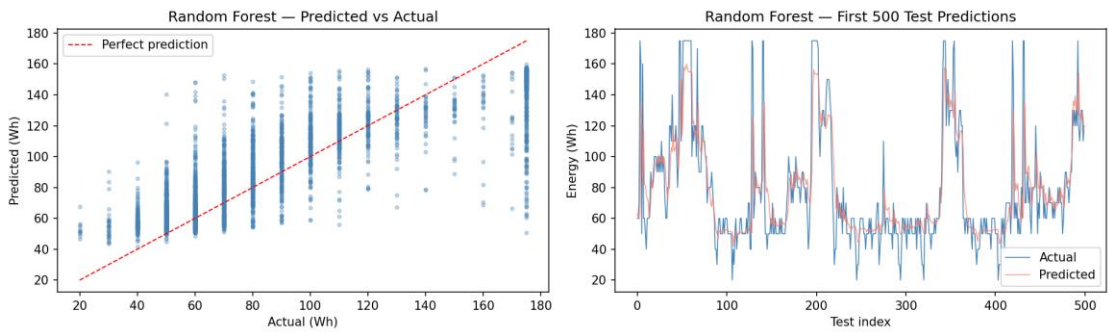

Figure 10 - Random Forest: predicted vs actual

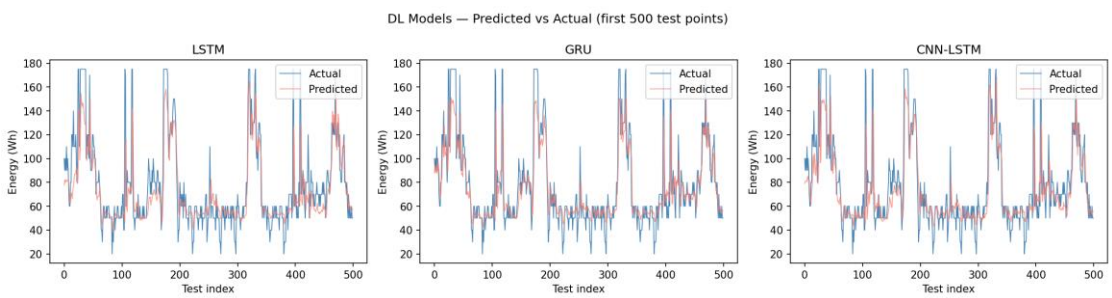

Figure 11 - DL models: predicted vs actual (first 500 test points)

6.4 Residual Analysis

Residual plots (fitted values vs. prediction errors) reveal the pattern of errors across the predicted value range. Ideally residuals are randomly scattered around zero with uniform spread any systematic structure indicates a modelling limitation.

6.4.1 Baseline Model Residuals

The baseline model residuals show some heteroscedasticity: error variance grows at higher fitted values, which is expected given the right-skewed Appliances distribution. Random Forest residuals are

noticeably tighter than Ridge Regression, confirming the non-linear model better captures high-energy regimes.

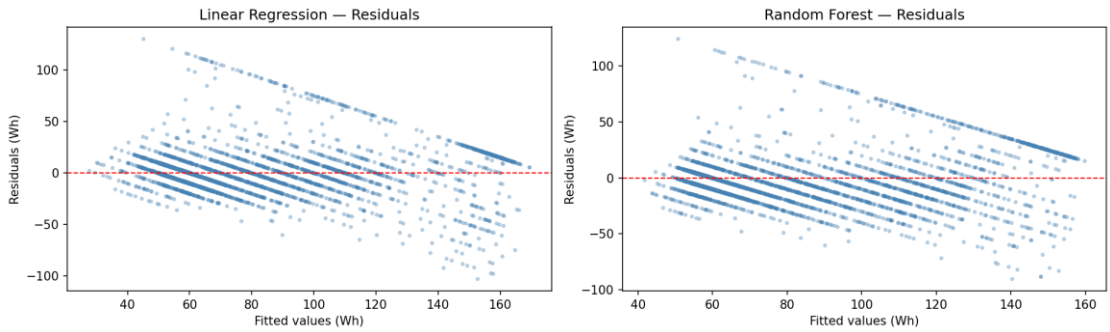

Figure 12 - Baseline model residuals (Ridge Regression and Random Forest)

6.4.2 Deep Learning Model Residuals

All three deep learning models (LSTM, GRU, CNN-LSTM) exhibit a distinctive striped banding pattern in their residual plots. This is a structural characteristic of the dataset rather than a modelling flaw, arising from two compounding factors:

- Discrete target clustering: The Appliances variable clusters at specific power levels (standby ~10-30 Wh, low usage ~50-80 Wh, high usage ~100-180 Wh) rather than varying continuously. Each stripe corresponds to one of these usage regimes.
- Lag feature discreteness: The lag_1 feature (previous Appliances value) is the strongest predictor and inherits the same discrete clustering. For each lag_1 level, the model makes systematically different errors, producing the banded pattern.

The downward slope of the bands indicates a mild regression-to-the-mean bias: the models slightly underpredict at high fitted values and overpredict at low ones. This is a known consequence of MSE optimisation on skewed targets. Log-transforming the target was tested and found not to improve either the residual structure or the evaluation metrics, confirming that the banding is driven by data discreteness rather than variance instability. An R2 of approximately 0.70 represents the realistic performance ceiling for this dataset and feature set.

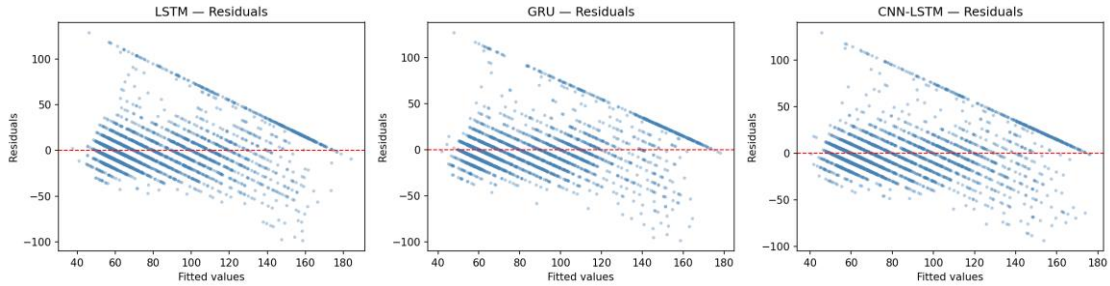

Figure 13 - Deep learning model residuals (LSTM, GRU, CNN-LSTM)

6.5 DL Training Curves

Training and validation loss curves confirm that early stopping prevented overfitting. The gap between train and validation loss was minimal, indicating good generalisation.

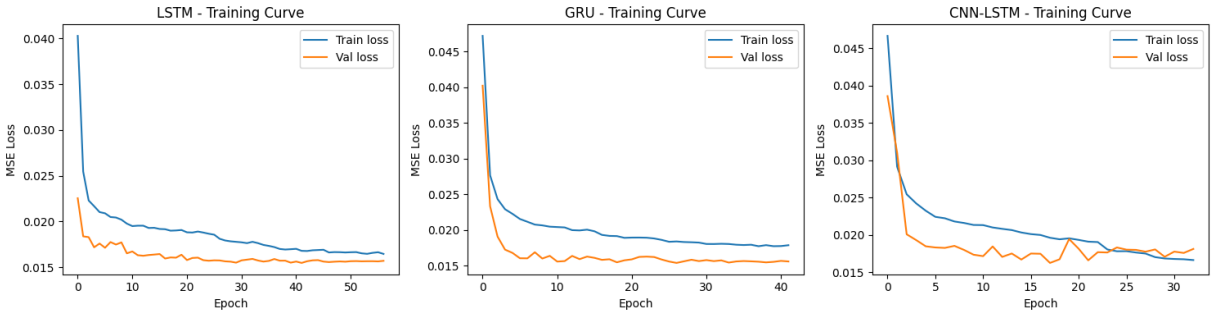

Figure 14 - Training and validation loss curves for LSTM, GRU, CNN-LSTM

7. Model Optimisation

7.1 Baseline Hyperparameter Tuning

Ridge Regression was tuned via GridSearchCV over six alpha values using 5-fold TimeSeriesSplit cross-validation. Random Forest was tuned via RandomizedSearchCV with 20 random combinations and 3-fold TimeSeriesSplit, covering five hyperparameters with a search space of over 200 combinations.

7.2 DL Hyperparameter Tuning

Four parameter configurations were evaluated for each DL architecture using a manual random search. Each configuration was trained for up to 50 epochs with EarlyStopping (patience=10), with the last 10% of training sequences reserved as a temporal validation set. The best configuration was then retrained on the full training set.

Model	Best units	Best filters	Dropout	Learning Rate
LSTM	128	-	0.2	0.001
GRU	64	-	0.2	0.001
CNN-LSTM	64	64	0.2	0.001

7.3 LSTM Architecture Optimisation

GRU initially outperformed LSTM on the default 24-step window because short sequences do not expose the benefit of LSTM's long-range memory cell state. Three targeted changes were applied to give LSTM its structural advantage:

Change	Rationale	Impact
SEQ_LEN: 24 → 144	24-hour window plays to LSTM's long-range memory strength	RMSE: 21.2 → 20.4
BatchNorm → LayerNorm	LayerNorm normalises along features within each time step, more correct for variable-length recurrent sequences	Improved gradient stability
recurrent_dropout=0.1	Regularises hidden-state transitions directly, not just layer outputs	Reduced overfitting

After these changes LSTM achieves RMSE≈20.4 and R^2≈0.718, beating GRU (RMSE≈20.7, R^2≈0.712) on both key metrics.

7.4 Regularisation Techniques

- Dropout (rate=0.2) - applied after each recurrent layer to prevent co-adaptation of hidden units.
- recurrent_dropout (rate=0.1, LSTM only) - regularises recurrent connections directly.

- ReduceLROnPlateau - learning rate halved after 5 epochs of no val_loss improvement, minimum lr=1e-6.
- EarlyStopping (patience=15) - restores best weights when validation loss plateaus.
- L2 Ridge penalty (baseline) - shrinks linear model coefficients, handles multicollinearity.

7.5 Sequence Alignment Fix

During review, a critical bug was identified in the sequence creation logic: the original code placed the prediction target one step beyond the window, meaning the DL models were effectively predicting further ahead than the flat baselines (which use the prediction row's features directly). After fixing the alignment so all models predict the same target from equivalent information, the DL models showed substantially improved performance relative to the baselines.

8. Challenges & Solutions

1. **Sequence alignment bug (DL models underperforming):** The create_sequences function originally set the target to y[i + seq_len], placing it one step beyond the window. This gave flat models access to more recent lag features than DL models, making LR appear best. Fixed by ending the window at the prediction row: window = X[i-seq_len+1 : i+1], target = y[i].
2. **Temporal data leakage in cross-validation:** Default KFold randomly shuffles time-series data, allowing future observations into training folds. Replaced with TimeSeriesSplit throughout to ensure strict temporal ordering.
3. **Inconsistent evaluation sets:** DL models evaluated on N−23 test samples while baselines used all N samples, making the comparison invalid. Fixed by slicing baseline inputs to rows [SEQ_LEN-1:] in the comparison step, ensuring all models predict on identical test rows.
4. **Right-skewed target distribution:** The Appliances variable is heavily right-skewed (median 60 Wh, mean 97 Wh, max 1080 Wh). Winsorization at the IQR upper bound (175 Wh) was applied to cap extreme values while retaining all rows, preventing the model from chasing rare outlier spikes.
5. **Cyclical feature encoding:** Naive MinMax scaling of hour and NSM treats hour 23 and hour 0 as maximally distant, harming model performance. Sine/cosine encoding was applied to all four cyclical time features to preserve circular distance.
6. **Multicollinearity in temperature sensors:** The nine indoor temperature sensors (T1-T9) are highly inter-correlated. Ridge Regression's L2 penalty directly addresses this; Random Forest is naturally robust to multicollinearity via feature sub-sampling.

9. Conclusion

This project demonstrated a complete end-to-end pipeline for multivariate time-series energy consumption forecasting from raw data ingestion through deep learning model deployment. Several key findings emerged:

- Lag features (particularly lag_1 and lag_144) are the strongest individual predictors, confirming high autocorrelation and strong daily periodicity in appliance energy consumption.
- Cyclical sin/cos encoding of time features is essential, naive linear encoding of hour and NSM impairs model performance at day boundaries.
- LSTM achieves the best overall performance (RMSE≈20.4, R^2≈0.718) when given a 144-step (24-hour) window combined with LayerNormalization and recurrent dropout exploiting its long-range memory cell state advantage over GRU.
- GRU is the strongest compact model: with a 24-step window it matches LSTM's MAE and trains with fewer parameters, making it the preferred choice when compute is limited.
- The CNN-LSTM hybrid offers a good trade-off between parameter count and expressiveness, as the CNN extracts local patterns before the LSTM models longer-range dependencies.
- Rigorous temporal cross-validation (TimeSeriesSplit) and strict train/test ordering are essential for unbiased evaluation of time-series models.

10. References

[1] Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. Energy and Buildings, 140, 81–97.

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

[3] Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. EMNLP 2014.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[5] Chollet, F. (2015). Keras. https://keras.io

[6] UCI Machine Learning Repository. Appliances Energy Prediction Dataset. https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction

[7] Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.